

AGENTDIAG VERSION 2

A plain-English guide to the project, its files, its results, and what to say in your review.

184

traces audited

48 / 91

hybrid
grouped-fold hits

6 / 6

automated tests
pass

PROJECT REVIEW EDITION • 29 SEPTEMBER 2026

Easy English. Honest scores. File-by-file explanations. Review-ready answers.

1. The project in one minute

Start with this page if you have to explain the project quickly.

AgentDiag v2 studies failures in teams of AI agents. It reads a conversation trace, estimates which agent was most responsible, points to the message where the failure happened, and prepares evidence for a human reviewer.

Simple summary: it is a tool that helps a person investigate an AI-agent failure. It gives a ranked suggestion and evidence; it does not prove who caused the failure.

What the system can do

Capability	In everyday words	Current status
Agent attribution	Rank the agents from most to least likely to be responsible.	Implemented; Bayesian model and hybrid are available.
Step attribution	Point to the message number most likely to contain the decisive mistake.	Implemented; separate StepFinder-trained ranker.
Evidence review	Show the message excerpts and a two-agent shortlist for a person to inspect.	Implemented; source indices are checked.
Missing logs	Still estimate responsibility if some agents' messages are absent.	Implemented and measured.
Early warning	Try to warn before a mistake happens.	Implemented, but results are not reliable yet.
Multimodal analysis	Understand image, audio, or video evidence.	Not demonstrated: supplied labeled traces have no such fields.

Key terms in simple English

Term	Simple meaning
Trace	The saved conversation and actions from one run of the agents.
Agent	One AI worker in the team.
Culprit label	The dataset's recorded answer about which agent made the failure.
Raw step	The original message number in the trace.
Accuracy	The share of cases where the model's top answer matches the saved answer.
Question-group split	Similar cases from the same question are kept together so they cannot leak across training and test.
Calibration	Whether a stated confidence level roughly matches how often predictions are correct.
Hybrid	A weighted combination of the Bayesian model and the external step ranker.

2. What happens to one trace?

The model only needs observed messages at prediction time. Saved answer labels are used for training and scoring, not as prediction inputs.

Step	Action	What it means
01	Read	Load the JSON conversation trace and its agent names.
02	Extract clues	Count simple signs such as errors, uncertainty, and corrections.
03	Estimate	Compute a probability for each possible responsible agent.
04	Locate	The step ranker scores each observed message as a possible failure step.
05	Explain	Build a review packet with ranked agents, message excerpts, and uncertainty.
06	Check	Compare predictions with recorded labels in the evaluation scripts.

Training and evaluation

The project uses 184 saved Who&When traces. Related traces are grouped by their question before splitting: 134 train, 27 validation, and 23 test traces. For the main four-agent task, the split has 65 train, 13 validation, and 13 test traces. The small test set is already inspected, so it is not a fresh independent confirmation.

The separate external step-ranker experiment uses 2,004 filtered StepFinder training traces from 58 questions. The training questions are screened against benchmark questions for close overlap. It selects its regularization setting using five folds that keep whole questions together.

At prediction time, the code parses the trace without reading the `mistake_agent` or `mistake_step` answer fields. The answer labels are needed only to train or evaluate the models.

Why keep question groups together?

If nearly identical examples appear in both training and test, a model may look accurate by recognizing familiar questions. Grouping related examples makes the evaluation more honest.

How this differs from Who&When

Who&When did run multi-agent systems to build its benchmark: it used automatically generated systems and a hand-crafted Magnetic-One system on task queries, saved failed runs, and had human experts label the responsible agent and decisive step. Its paper describes 184 annotated failure tasks from 127 systems. Those labels let researchers test diagnosis methods.

For diagnosis testing, Who&When methods then analyzed the saved logs. Their paper tested judging the full log at once, reading the log step by step, and narrowing down the step by binary search. The official repository accepts a benchmark data folder and runs one of these inference methods; it does not rerun every original agent for each prediction.

AgentDiag v2 follows that post-run analysis setup: we evaluated on stored traces and did not install or launch the 127 source systems. The single-trace commands can analyze a new saved JSON trace, but they are not connected to a live agent. We did not test image, audio, or video input. See the [Who&When paper](#) and [official benchmark repository](#).

3. The project phases, from start to finish

Each phase has a human-readable report and machine-readable output files.

Phase	What was done	Main outputs
0 • Protocol	Audit all 184 traces; make fixed question-safe train/validation/test splits.	artifacts/phase0_protocol/Phase0_Results.md; audit.json; splits.json
1 • Parsing	Read messages and labels; flag label mismatches and summarize agent interactions.	artifacts/phase1_parsing/Phase1_Results.md; clean_records.jsonl; label_step_exceptions.json
2 • Baselines	Measure simple rules before using a more involved model.	artifacts/phase2_evidence_baselines/Phase2_Results.md; baselines.json; features.jsonl
3 • Bayesian model	Learn agent probabilities from error, uncertainty, and correction clues.	artifacts/phase3_bayesian_inference/Phase3_Results.md; model.json; validation_predictions.jsonl
4 • Missing logs	Hide some logs and test whether the system can still make a prediction.	artifacts/phase4_missing_evidence/Phase4_Results.md; metrics.json; missing_evidence_predictions.jsonl
5 • Calibration	Adjust confidence using validation data, then score on held-out cases.	artifacts/phase5_calibration_evaluation/Phase5_Results.md; calibration.json; predictions; reliability_diagram.png
6 • Early warning	Try an HMM-based time-aware warning model and measure false alarms.	artifacts/phase6_temporal_hmm/Phase6_Results.md; hmm_model.json; metrics.json; test_warnings.jsonl
7 • Modality audit	Check whether the data really contains labeled image/audio/video inputs.	artifacts/phase7_multimodal_scope/Phase7_Results.md; modality_audit.json
8 • Final report	Summarize results and check which presentation claims the evidence supports.	artifacts/phase8_final_report/Final_Report.md; PPT_Alignment.md; final PPT; verification
9 • Evidence packets	Give ranked agents, message quotes, and model sensitivity for human review.	artifacts/phase9_evidence_triage/Phase9_Results.md; metrics.json; test_review_packets.jsonl

A working feature is not automatically a successful feature. For example, early warning is implemented, but its false-alarm results are poor.

4. Model choices in everyday language

The project keeps multiple approaches so its stronger claims can be checked against simple baselines.

Method	Plain-English explanation	What we learned
Position baseline	Usually prefer an agent based on its position in the team.	4/13 on the four-agent test; simple reference point.
Keyword baseline	Look for the first obvious error-like wording.	3/13; did not solve the task.
Bayesian network (BN)	Combine several clues and update the chance for each agent.	5/13 top choice; probabilities can be calculated exactly.
Text ranker	Learn patterns from words and handoffs between agents.	33/91 in grouped folds, below BN's 40/91 for top-choice accuracy.
External step ranker	Learn which message and agent resemble failure examples from StepFinder.	Strong raw-step score relative to older paper values; agent scores vary by subset.
Hybrid	Mix the BN probability with the external ranker probability.	48/91 vs BN 40/91 in grouped folds; promising, not conclusive.
HMM early warning	Track clue patterns over time and try to warn before the recorded failure.	Only 2/10 eligible cases warned for the right agent; many wrong-agent warnings.

What does the review packet add?

It shows the likely agents, their probabilities, a two-agent shortlist, and original message excerpts that a reviewer can find in the trace. It also removes one agent's evidence in turn and shows how the model changes. That last step is a model sensitivity check; it does not prove real-world cause and effect.

Why not use the semantic and lexical models as the final answer?

Some configurations looked strong on validation but fell to 3/13 on the inspected test. The text ranker also scored 33/91 in grouped folds versus 40/91 for the BN. Those outcomes are kept as failed experiments instead of being presented as improvements.

5. The new-version timeline

This is what changed as AgentDiag v2 was built and then improved.

Stage	What was completed
Start fresh	Created a separate agentdiag_v2 folder. The new pipeline does not import old phase scripts or results.
Plan the data	Audited the dataset and fixed train/validation/test splits by question group.
Build a trustworthy baseline	Implemented parsing, baselines, a categorical Bayesian model, calibration, missing-log handling, and phase-by-phase outputs.
Review the feature claims	Checked the presentation scope. The project supports multi-agent failure analysis; the supplied labeled traces do not support multimodal performance claims.
Make outputs reviewable	Added a shortlist, evidence excerpts, source-message indices, and sensitivity checks for human review.
Try accuracy improvements	Tested keyword, lexical, embedding, and external StepFinder-based methods; recorded both gains and failures.
Tune the external step ranker	Used five grouped folds over external questions to choose $L2=0.45$. Archived the previous $L2=1.0$ model and predictions.
Combine models	Updated the hybrid and measured it against the BN on nested question-group folds.
Verify and explain	Recomputed stored results, ran 17 pipeline checks, 13 upgrade checks, and six automated tests; updated project reports.

Implementation versus research result

Implementation succeeded: the code and outputs run and pass the recorded checks. Accuracy improvement is narrower: step accuracy increased modestly over the previous step-ranker setting, and the hybrid beat the BN in grouped folds. The published AgenTracer benchmark was not beaten.

6. Accuracy results at a glance

Top-choice agent accuracy, exact raw-step accuracy, and joint accuracy are different measurements. Do not mix them.

Model / evaluation	Agent result	Step result	Meaning
BN, four-agent test	5/13 · 38.5%	Not measured here	Small, already-inspected test.
Hybrid, four-agent test	6/13 · 46.2%	Step model also gives a step	Exploratory; same 13 cases were inspected earlier.
BN, grouped folds	40/91 · 44.0%	Not measured here	Question-group out-of-fold baseline.
Hybrid, grouped folds	48/91 · 52.7%	Not the step benchmark	+8.8 percentage points vs BN; paired $p=0.077$.
Tuned step model, Algorithm-Generated	62/126 · 49.2%	36/126 · 28.6%	Joint correct: 33/126.
Tuned step model, Hand-Crafted	37/58 · 63.8%	10/58 · 17.2%	Joint correct: 8/58.
Tuned step model, combined	99/184 · 53.8%	46/184 · 25.0%	Joint correct: 41/184.

What changed against the archived L2=1.0 step ranker?

Subset	Measure	Before	Now (L2=.45)	Change
Algorithm-Generated	Agent	63/126	62/126	–1 case
Algorithm-Generated	Step	34/126	36/126	+2 cases
Hand-Crafted	Agent	36/58	37/58	+1 case
Hand-Crafted	Step	9/58	10/58	+1 case

The absolute improvements are small. The new setting makes four additional step predictions correct across the two benchmark subsets, while agent accuracy changes by only one trace down/up. This is not a large across-the-board accuracy jump.

On the hybrid's 91 grouped cases, the accuracy rose from 44.0% to 52.7%. The exact paired p-value is 0.077, above the common 0.05 cutoff. Say “promising improvement, not conclusive,” not “proven improvement.”

7. Which published benchmark did we beat?

Short answer: we are close to some older Who&When numbers, but the protocols differ. We did not beat AgenTracer overall.

Benchmark / subset	Published agent / step	Our agent / step	Plain result
Older Who&When • generated	51.12% / 13.53%	49.2% / 28.6%	Agent is 1.9 points lower; step is higher.
Older Who&When • crafted	53.44% / 3.51%	63.8% / 17.2%	Our measured scores are higher on both.
AgenTracer • generated	63.73% / 37.30%	49.2% / 28.6%	Our scores are lower by 14.5 / 8.7 points.
AgenTracer • crafted	63.82% / 20.68%	63.8% / 17.2%	Agent is almost tied (0.03 point lower); step is 3.4 points lower.
Who&When Pro	No verified score	Not evaluated	Official labeled evaluation set was unavailable.

Careful wording: The new model scores above the older paper's reported step rates and crafted-agent rate, and is close to its generated-agent rate. But the scoring methods differ, so this is not a fair claim that we beat Who&When. We are below AgenTracer on all four published score points, with a near tie on crafted-agent accuracy.

Why are the comparisons limited?

Our evaluation uses exact agent-name and raw message-index matching on 184 Who&When traces. The older paper used its own judge setup and permissive substring checks; AgenTracer also has its own method and evaluation details. We do not have the official Who&When Pro labeled set and scorer. A true head-to-head result needs the same examples, same labels, same scorer, and comparable access to the trace.

Benchmark sources

Who&When: [arXiv:2505.00212](#) • AgenTracer: [arXiv:2509.03312](#) • Who&When Pro: [official repository](#) • StepFinder: [official repository](#).

8. Novelty, strengths, and honest limits

The project's useful contribution is its review workflow and evidence trail—not a claim that a new algorithm has beaten the field.

What is useful and distinctive in this implementation?

It brings together probability-based attribution, explicit handling of missing logs, a ranked shortlist, quoted messages with original positions, and a model sensitivity check in one auditable review packet. It also preserves machine-readable predictions and validation reports so results can be recomputed.

What is not claimed as new?

Bayesian networks, confidence calibration, root-cause analysis, or step ranking are not new ideas by themselves. Earlier work already studies related methods. We have not established that this combination is publication-level novelty.

What did not work well?

Feature / experiment	Observed result	What to say
Early warning	2/10 eligible correct; 11/12 test traces got a wrong-agent warning.	Feature exists, but warning quality is not reliable.
BGE semantic ranker	9/13 on validation but 3/13 on test in an exploratory setting.	Validation overfit; not deployed.
Text ranker	33/91 vs BN 40/91 in grouped folds.	Did not improve top-one accuracy.
Multimodal claim	No supplied labeled image/audio/video trace fields.	Do not claim multimodal performance.
Benchmark leadership	Below AgenTracer on all published values except near tie on one measure.	Do not claim state of the art.

What data or resources would help next?

A fresh and larger labeled test set, an official matched benchmark and scoring script, reliable annotated step labels, and compute or model access for stronger reasoning models. The current run used local resources only; no GPU or model API was available.

9. File-by-file guide: top level and code

Paths below are relative to the agentdiag_v2 folder. JSON is structured data; JSONL is one structured record per line.

File	What it does
README.md	How to install, run, and understand the project.
ACCURACY_PUSH_PROMPT.md	Saved continuation instructions, current scores, and research guardrails.
ALL_ACCURACY_RESULTS.md	One table collecting baseline, phase, hybrid, and external benchmark scores.
BENCHMARK_STATUS.md	Plain-language status against Who&When, AgenTracer, and Who&When Pro.
RESEARCH_UPGRADE.md	Paper comparison, novelty claims, experiments that failed, and open gaps.
requirements.txt	Python packages needed to run the project.
init.py	Marks this directory as the AgentDiag v2 Python package.
.gitignore	Keeps generated caches and temporary files out of Git tracking.
run_pipeline.py	Runs the main phases and writes their reports and outputs.
predict.py	Makes a single-trace Bayesian diagnosis and review packet.
predict_step.py	Uses the external ranker to predict an agent and message step.
predict_hybrid.py	Uses the combined BN and step-ranker model for four-agent traces.
benchmark_step.py	Filters StepFinder training data, selects a setting by grouped CV, and evaluates exact predictions.
hybrid_upgrade.py	Runs nested question-group evaluation, chooses the blend weight, and saves hybrid metrics/model.
accuracy_study.py	Recreates the BN versus text-ranker grouped-fold comparison.
make_presentation.py	Builds the result-aligned project review slideshow.
create_project_review_pdf.py	Rebuilds this plain-language PDF review guide.
verify.py	Recomputes core artifact counts, metrics, and phase checks.
verify_upgrade.py	Rechecks step and hybrid predictions, metrics, folds, and probabilities.
src/data.py	Loads and parses traces; creates train/test splits and label-free trace views.
src/bayes.py	Fits categorical probabilities and performs Bayesian inference with variable elimination.
src/evaluation.py	Computes baselines, scores, calibration, and evaluation plots.
src/ranker.py	Implements the exploratory text and handoff ranker.
src/step_ranker.py	Implements the sparse model that predicts a responsible agent and raw message step.
src/temporal.py	Implements the HMM-style time-aware warning model.
src/triage.py	Creates source-grounded human review packets and evidence excerpts.
tests/test_pipeline.py	Tests core pipeline behavior, output contracts, and the generated presentation.
tests/test_step_ranker.py	Tests the step-ranker prediction interface and expected output.

File	What it does
tests/test_hybrid.py	Tests that the hybrid can produce a valid diagnosis.

10. File-by-file guide: reports, models, and outputs

This appendix covers the non-code outputs too, so you can point to the evidence behind a statement.

File or folder	What it contains
data/stepfinder_safe_questions.json	Approved question hashes, source revision, and filtering method for external training.
artifacts/phase_manifest.json	Index of the phase reports and output folders.
artifacts/phase0_protocol/Phase0_Results.md	Human-readable data-split report.
artifacts/phase0_protocol/audit.json	Machine-readable dataset audit and counts.
artifacts/phase0_protocol/splits.json	Saved trace IDs assigned to train, validation, and test.
artifacts/phase1_parsing/Phase1_Results.md	Parsing results and label consistency summary.
artifacts/phase1_parsing/clean_records.jsonl	Parsed trace records used by later phases.
artifacts/phase1_parsing/interaction_graph_summary.json	Descriptive agent-to-agent interaction counts.
artifacts/phase1_parsing/label_step_exceptions.json	Cases where culprit label and raw mistake-step speaker do not align.
artifacts/phase2_evidence_baselines/Phase2_Results.md	Baseline accuracy results.
artifacts/phase2_evidence_baselines/baselines.json	Machine-readable baseline metrics.
artifacts/phase2_evidence_baselines/features.jsonl	Extracted trace and agent evidence features.
artifacts/phase3_bayesian_inference/Phase3_Results.md	Bayesian model selection and inference checks.
artifacts/phase3_bayesian_inference/model.json	Saved Bayesian model parameters.
artifacts/phase3_bayesian_inference/validation_predictions.jsonl	Validation predictions used for model choices.
artifacts/phase4_missing_evidence/Phase4_Results.md	Plain-language missing-log experiment results.
artifacts/phase4_missing_evidence/metrics.json	Accuracy and probability scores for each missing-log scenario.
artifacts/phase4_missing_evidence/missing_evidence_predictions.jsonl	Per-trace predictions under the saved missing-log masks.
artifacts/phase5_calibration_evaluation/Phase5_Results.md	Held-out accuracy and confidence-calibration report.
artifacts/phase5_calibration_evaluation/calibration.json	Temperature and calibration metrics.
artifacts/phase5_calibration_evaluation/reliability_diagram.png	Plot comparing predicted confidence with observed correctness.
artifacts/phase5_calibration_evaluation/test_predictions.jsonl	Per-case predictions on four-agent test traces.
artifacts/phase5_calibration_evaluation/all_agent_counts_test_predictions.jsonl	Predictions for test traces with different team sizes.
artifacts/phase5_calibration_evaluation/exploratory_agent_count_metrics.json	Small-sample score breakdown by number of agents.

File or folder	What it contains
artifacts/phase6_temporal_hmm/Phase6_Results.md	Early-warning experiment explanation and measured results.
artifacts/phase6_temporal_hmm/hmm_model.json	Saved temporal model parameters.
artifacts/phase6_temporal_hmm/metrics.json	Warning accuracy and false-alarm metrics.
artifacts/phase6_temporal_hmm/test_warnings.jsonl	Per-trace time-aware warnings.
artifacts/phase7_multimodal_scope/Phase7_Results.md	States which modality claims the supplied data can support.
artifacts/phase7_multimodal_scope/modality_audit.json	Machine-readable count of image/audio/video fields.
artifacts/phase8_final_report/Final_Report.md	End-to-end summary and limitations.
artifacts/phase8_final_report/PPT_Alignment.md	Maps presentation claims to implemented evidence.
artifacts/phase8_final_report/AgentDiag_v2_Final_Review.pptx	Generated presentation with claims aligned to measured results.
artifacts/phase8_final_report/summary.json	Machine-readable headline scores and split sizes.
artifacts/phase8_final_report/verification.json	Pass/fail status and checks for the main pipeline.
artifacts/phase8_final_report/Verification_Report.md	Human-readable main-pipeline verification results.
artifacts/phase9_evidence_triage/Phase9_Results.md	Shortlist and evidence-packet results.
artifacts/phase9_evidence_triage/metrics.json	Top-one and top-two shortlist coverage numbers.
artifacts/phase9_evidence_triage/test_review_packets.jsonl	One source-grounded review packet per test trace.
accuracy_study/Accuracy_Study.md	Explains why the text ranker was not selected over the BN.
accuracy_study/results.json	Grouped-fold BN and text-ranker metrics.
accuracy_study/group_fold_predictions.jsonl	Per-trace predictions from the grouped-fold accuracy study.
benchmark_step/Benchmark_Report.md	Current tuned model benchmark and fair-comparison caveats.
benchmark_step/step_model.json	Current L2=0.45 external step-and-agent model.
benchmark_step/metrics.json	Full benchmark, confidence intervals, and CV results.
benchmark_step/full_benchmark_predictions.jsonl	Prediction and answer per Who&When benchmark trace.
benchmark_step/external_training_cv_metrics.json	Scores used to select among L2 values.
benchmark_step/external_training_folds.json	Question-to-fold assignment and trace membership.
benchmark_step/external_cv_predictions_l2_0_25.jsonl	External grouped-fold predictions for L2 0.25.
benchmark_step/external_cv_predictions_l2_0_35.jsonl	External grouped-fold predictions for L2 0.35.
benchmark_step/external_cv_predictions_l2_0_45.jsonl	External grouped-fold predictions for selected L2 0.45.
benchmark_step/provenance.json	Training IDs hash, source revision, filtering, and selected setting.
benchmark_step/previous_l2_1/step_model_l2_1.json	Archived previous external model, L2=1.0.
benchmark_step/previous_l2_1/full_benchmark_predictions_l2_1.jsonl	Archived prior-model predictions used for the before/after comparison.

File or folder	What it contains
hybrid_upgrade/Hybrid_Report.md	Current hybrid score and paired BN comparison.
hybrid_upgrade/hybrid_model.json	Saved BN plus tuned step-model blend and weight.
hybrid_upgrade/metrics.json	Grouped-fold, paired, and exploratory test metrics.
hybrid_upgrade/nested_fold_predictions.jsonl	Out-of-fold BN and hybrid probabilities for 91 traces.
hybrid_upgrade/test_predictions.jsonl	Exploratory predictions on the previously inspected 13-case test.
hybrid_upgrade/verification.json	Machine-readable checks for step/hybrid outputs.
hybrid_upgrade/Verification_Report.md	Human-readable verification checks for the upgrade.
hybrid_upgrade/previous_step_model_I2_1/	Folder holding the prior hybrid report, metrics, predictions, verification, model, and its L2=1.0 dependency.
hybrid_upgrade/previous_step_model_I2_1/Hybrid_Report.md	Archived human-readable report for the previous hybrid.
hybrid_upgrade/previous_step_model_I2_1/hybrid_model.json	Archived prior hybrid model settings and reference to its archived step model.
hybrid_upgrade/previous_step_model_I2_1/metrics.json	Archived metrics for the prior hybrid run.
hybrid_upgrade/previous_step_model_I2_1/nested_fold_predictions.jsonl	Archived fold-by-fold predictions from the prior hybrid run.
hybrid_upgrade/previous_step_model_I2_1/step_model_I2_1.json	Archived dependency model used by the prior hybrid.
hybrid_upgrade/previous_step_model_I2_1/test_predictions.jsonl	Archived per-trace predictions for the old 13-case test evaluation.
hybrid_upgrade/previous_step_model_I2_1/verification.json	Archived pass/fail verification details for the prior hybrid.
AgentDiag_v2_Project_Review_Guide.pdf	This designed, plain-language project and review guide.

Generated cache note

Python `__pycache__` folders and artifacts/.matplotlib/fontlist-v390.json are generated caches. They are not project logic, model evidence, or required review deliverables.

11. How to run and demonstrate it

Use the commands from the repository root. The exact data paths assume the project folder layout shown in this workspace.

Purpose	Command
Run all main phases	<code>python3 agentdiag_v2/run_pipeline.py</code>
Verify generated phase outputs	<code>python3 agentdiag_v2/verify.py</code>
Run automated tests	<code>python3 -m pytest agentdiag_v2/tests -q</code>
Check step/hybrid benchmark outputs	<code>python3 -m agentdiag_v2.verify_upgrade</code>
Predict a single trace	<code>python3 agentdiag_v2/predict.py</code> <code>'Agents_Failure_Attribution/Who&When/Algorithm-Generated/1.json'</code>
Predict step and agent	<code>python3 agentdiag_v2/predict_step.py</code> <code>'Agents_Failure_Attribution/Who&When/Algorithm-Generated/1.json'</code>
Predict with hybrid	<code>python3 agentdiag_v2/predict_hybrid.py</code> <code>'Agents_Failure_Attribution/Who&When/Algorithm-Generated/1.json'</code>

What was checked in this run?

The main artifact verification passed 17 checks; the step/hybrid verification passed 13 checks; six automated tests passed. The step and hybrid command-line predictors also returned valid JSON for a sample trace.

Reproduce the tuned external benchmark

The saved training source was StepFinder commit **48d71c7090adca3667a31b043a78484f407b4a2f**. With that data available locally, run **`python3 agentdiag_v2/benchmark_step.py --stepfinder-data /path/to/StepFinder/data`**, then rerun the hybrid and verification commands. The official source release and exact filtering configuration are recorded in the project.

For a live review demo, run the BN prediction first, show its ranked probabilities and cited excerpts, then show the hybrid result. Explain that the prediction helps a reviewer; it is not a proof of causation.

12. Questions you may be asked

These short answers are designed for a project review. Expand them in your own words if the examiner asks follow-up questions.

What problem does the project solve?

It helps identify which agent and which message may have contributed most to a failed multi-agent run, and gives a human evidence to inspect.

What does the model output?

It outputs probabilities for the agents, a likely raw message step, and a review packet with a shortlist and trace excerpts.

What is the main model?

A categorical Bayesian network that combines error, uncertainty, and correction clues. A separate external step ranker can be blended with it.

Why use Bayesian inference?

It gives a probability for each possible agent and can still calculate when some observations are missing.

What does the hybrid do?

It takes a weighted average of the Bayesian network and the external model's agent probabilities. The weight is selected inside training folds.

How much data was used?

The supplied benchmark has 184 traces. The main four-agent split has 65 training, 13 validation, and 13 test traces. The external ranker was trained on 2,004 filtered StepFinder traces from 58 questions.

What is your best measured agent accuracy?

The hybrid got 48 of 91, or 52.7%, across question-group folds; the Bayesian model got 40 of 91, or 44.0%. The paired p-value is 0.077, so the result is not conclusive.

Did you beat the old Who&When paper?

Some of our exact scores are higher, especially step accuracy, but our scoring protocol differs. I cannot fairly claim we beat it.

Did you beat AgenTracer?

No. We are below its published agent and step rates on the generated subset. Our crafted-agent score is almost equal, but slightly lower by 0.03 percentage points.

Why is the hybrid p-value important?

It checks whether the improvement is convincing when comparing the same held-out cases. A p-value of 0.077 is suggestive, but above the common 0.05 threshold.

Is the 13-case test result reliable?

It is useful as a small result, but it was inspected during earlier work. It is exploratory, not a fresh untouched test.

Does the project support missing logs?

Yes. Missing evidence is marginalized instead of being treated as if it were clean evidence. On this small test, results vary with how many logs are hidden.

Does the project do multimodal analysis?

No multimodal performance claim is supported. The supplied labeled traces have no image, audio, or video fields.

Is early warning reliable?

Not yet. The HMM warned for the right agent in 2 of 10 eligible cases and warned for the wrong agent in 11 of 12 held-out traces.

What is novel about your project?

The practical contribution is an auditable review packet combining probabilities, missing-log handling, a shortlist, source excerpts, and sensitivity checks. I do not claim the underlying methods are new or state of the art.

How did you prevent leakage?

Question groups stay together in splits and folds; external training questions are filtered for close overlap; prediction reads unlabeled traces. The original benchmark was still inspected in development, so it is not a pristine holdout.

What are the main limitations?

Small test sets, non-matched published comparisons, unreliable early warnings, no multimodal labels, no Who&When Pro score, and no independent proof of benchmark leadership.

What would you do next?

Freeze the method, obtain a larger unseen labeled benchmark and official scorer, evaluate all systems on the same cases, and improve warning quality with more reliable temporal labels.

Did the implementation work?

Yes. The saved predictions and metrics passed 17 artifact checks, 13 upgrade checks, six tests, and smoke runs of both specialized predictors. This confirms implementation, not high accuracy.

If you remember just one answer: “The system produces an evidence-backed suggestion for multi-agent failure review. The implementation works; accuracy improved in some measures, but the results are small and do not establish that we beat AgentTracer.”

13. Final review checklist

A short final page to prepare before you present.

Review point	What to say
Say what it is	A multi-agent failure-attribution and review-support tool.
Say what it is not	It is not a causal proof, not a reliable early-warning system, and not a demonstrated multimodal system.
Give the strongest result	Hybrid 48/91 vs BN 40/91 in grouped folds; paired $p=0.077$.
Give the external benchmark result	Tuned step model: generated 49.2% agent / 28.6% step; crafted 63.8% / 17.2%.
Describe benchmark standing accurately	Near or above some older Who&When rates under different scoring; below AgenTracer overall; no Pro score.
Name the evidence files	BENCHMARK_STATUS.md, ALL_ACCURACY_RESULTS.md, phase reports, and saved JSON/JSONL predictions.
Be open about limits	Small and reused test set; limited compute; no matched external scorer; multimodal labels absent.
Explain why the outputs matter	A reviewer can trace each suggestion back to a message in the original run.

Project files are under **agentdiag_v2/**. This guide was generated from the saved reports, results, model artifacts, and verification outputs in that folder.

Thank you

Be clear about the measured result, show where the evidence comes from, and explain the limitation before someone has to ask.